



동양미래대학교
DONGYANG MIRAE UNIVERSITY

딥러닝응용프로그래밍

학습 최적화와 일반화

고수진

Generalization

학습은 주어진 데이터에서 손실이 줄어들도록 파라미터를 바꾸는 과정임.

일반화는 학습에 사용하지 않은 데이터에도 예측이 잘 적용되는 성질임.

학습 성능만으로 새 데이터의 성능을 판단할 수 없음.

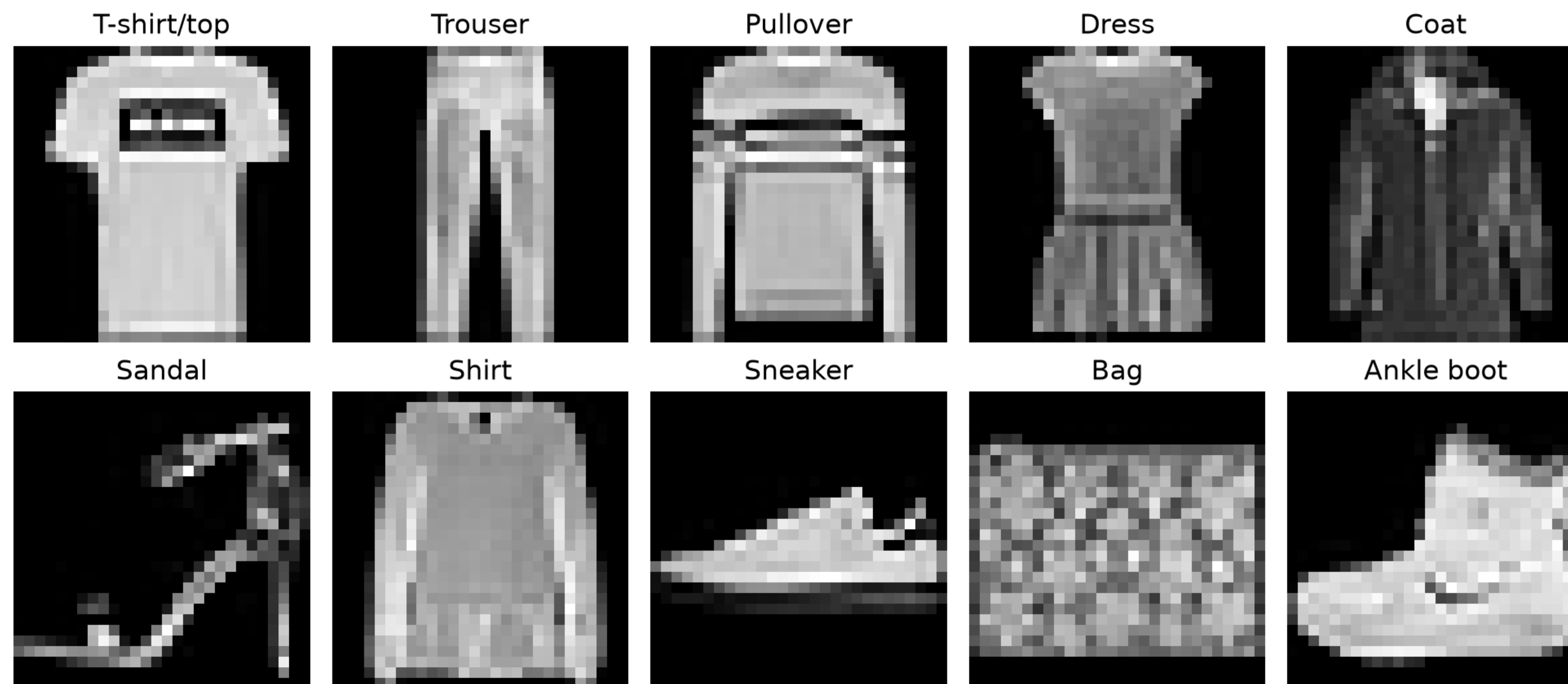
먼저 "처음 보는 데이터에서도 잘하는가?"를 확인함.

Training / Validation / Test

데이터	사용 목적	주의점
Training	가중치를 갱신함	학습 성능만으로 선택하지 않음
Validation	설정.checkpoint를 선택함	선택을 반복하면 검증에도 맞춰질 수 있음
Test	선택 완료 후 최종 평가함	설정을 다시 고르는 데 사용하지 않음

분할은 모델을 학습하기 전에 고정함.

Fashion-MNIST



28×28 회색조 의류 이미지, 10개 클래스임.

학습 600장 / 검증 1,000장임.

공식 학습 60,000장에서 고정된 순서로 분리함.

작은 데이터로 학습·검증 차이를 관찰함.



Fashion-MNIST

```
1  import copy, json, random, time, platform
2  from pathlib import Path
3  import numpy as np
4  import torch
5  from torch import nn
6  from torch.utils.data import TensorDataset, DataLoader
7  from torchvision.datasets import FashionMNIST
8  import matplotlib.pyplot as plt
9  SEED, EPOCHS, BATCH, LR = 42, 160, 64, 0.05
10 DEVICE = torch.device('cpu')
11 torch.set_num_threads(2)
12 torch.use_deterministic_algorithms(True)
13 OUT = Path('results'); OUT.mkdir(exist_ok=True)
14 def seed_all(seed=SEED):
15     random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
16 seed_all()
17 print('Python:', platform.python_version(), 'PyTorch:', torch.__version__)
18 print('CPU / seed 42 / batch 64 / SGD lr 0.05 / 160 epochs')
```

```
Python: 3.13.15 PyTorch: 2.11.0+cpu
CPU / seed 42 / batch 64 / SGD lr 0.05 / 160 epochs
```



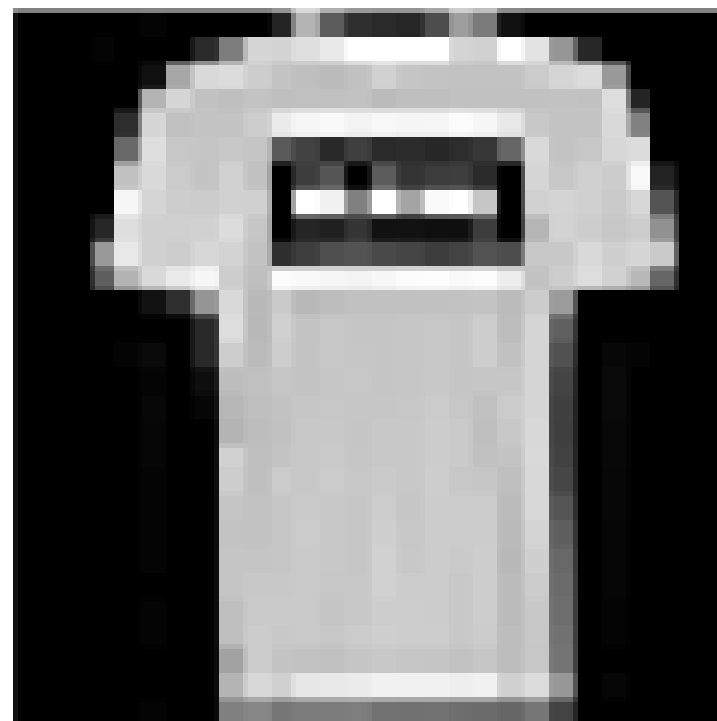
Fashion-MNIST

```
1 raw = FashionMNIST('./data', train=True, download=True)
2 ids = torch.randperm(len(raw), generator=torch.Generator().manual_seed(SEED))
3 train_ids, val_ids = ids[:600], ids[600:1600]
4 assert not set(train_ids.tolist()) & set(val_ids.tolist())
5 def subset(indices):
6     return TensorDataset(raw.data[indices].unsqueeze(1).float()/255,
7                           raw.targets[indices].long())
8 train_set, val_set = subset(train_ids), subset(val_ids)
9 def make_loader(ds, shuffle=False, batch_size=BATCH):
10     return DataLoader(ds, batch_size=batch_size, shuffle=shuffle, num_workers=0,
11                       generator=torch.Generator().manual_seed(SEED))
12 train_loader = make_loader(train_set, batch_size=256)
13 val_loader = make_loader(val_set, batch_size=256)
14 print('Train:', len(train_set), 'Validation:', len(val_set))
15 x, y = next(iter(make_loader(train_set)))
16 print('x:', x.shape, 'y:', y.shape)
17 fig, axes = plt.subplots(2, 5, figsize=(10, 4))
18 for label, ax in enumerate(axes.flat):
19     index = int((raw.targets == label).nonzero()[0])
20     ax.imshow(raw.data[index], cmap='gray', vmin=0, vmax=255)
21     ax.set_title(raw.classes[label], fontsize=9); ax.axis('off')
22 plt.tight_layout(); plt.show()
```

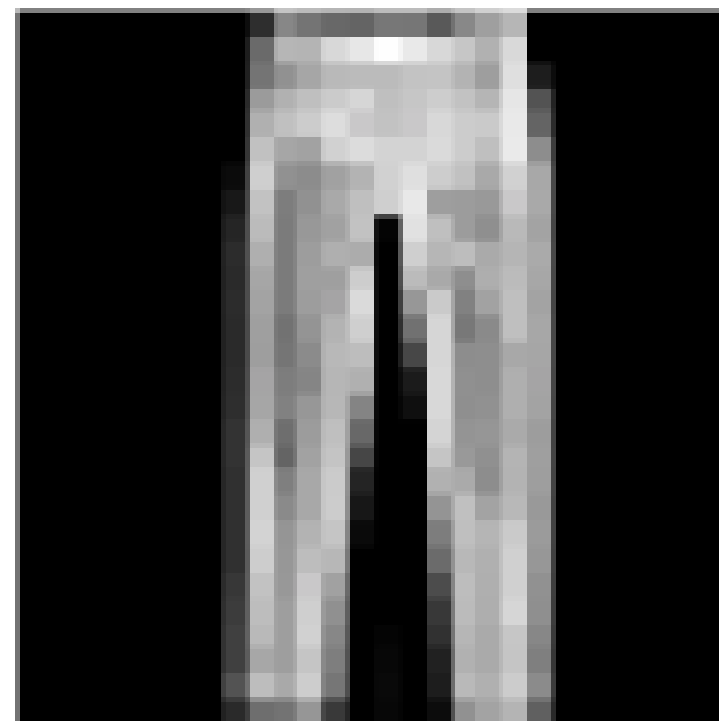
Fashion-MNIST

Train: 600 Validation: 1000
x: torch.Size([64, 1, 28, 28]) y: torch.Size([64])

T-shirt/top



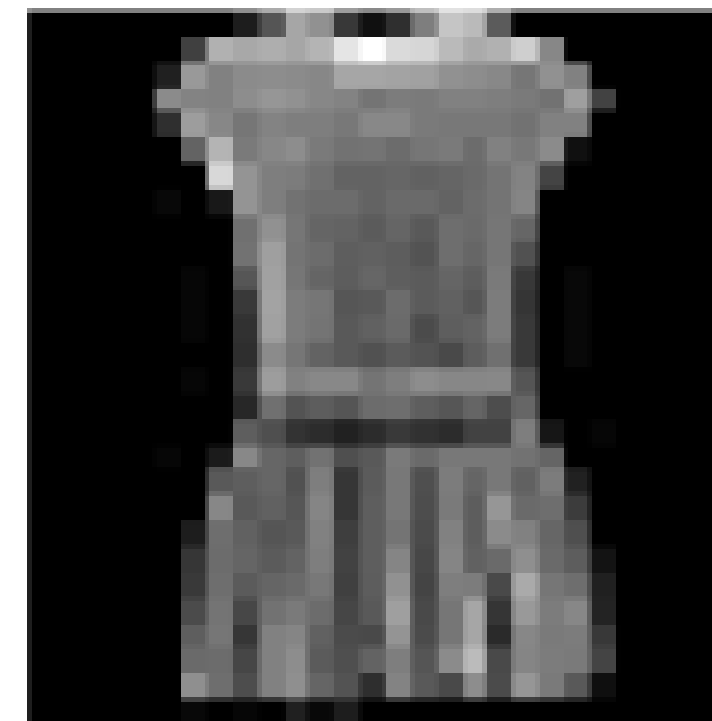
Trouser



Pullover



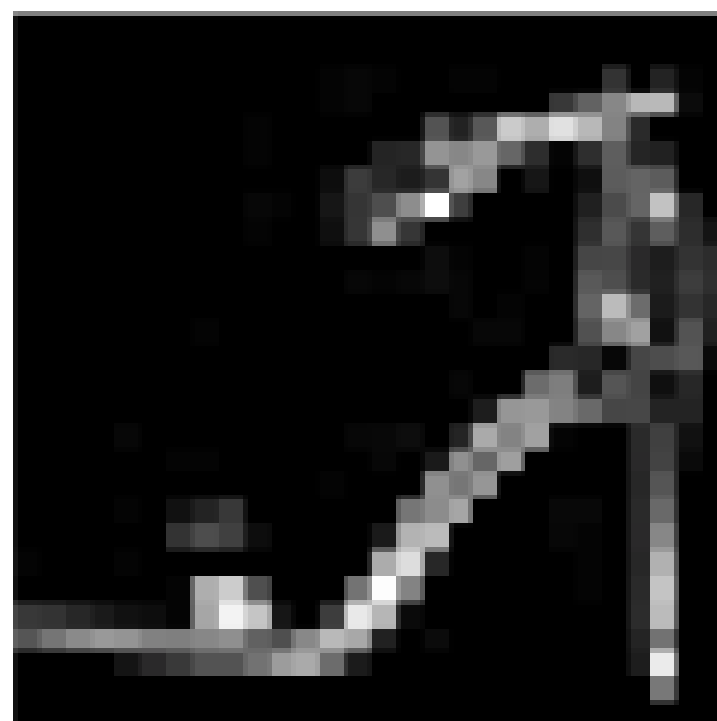
Dress



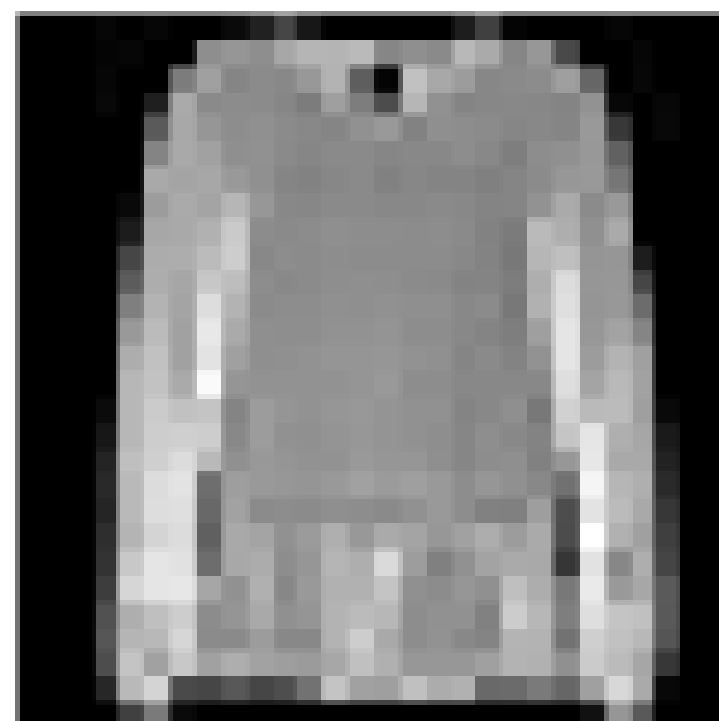
Coat



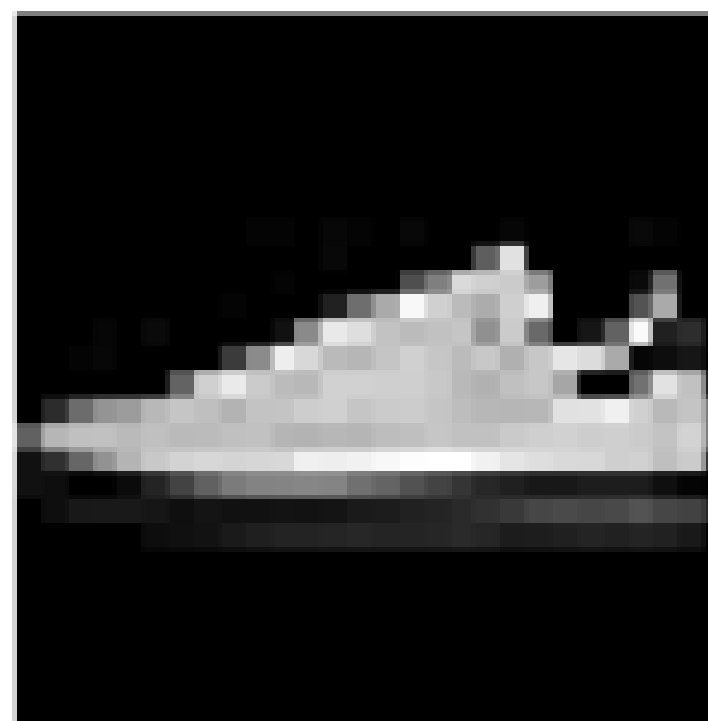
Sandal



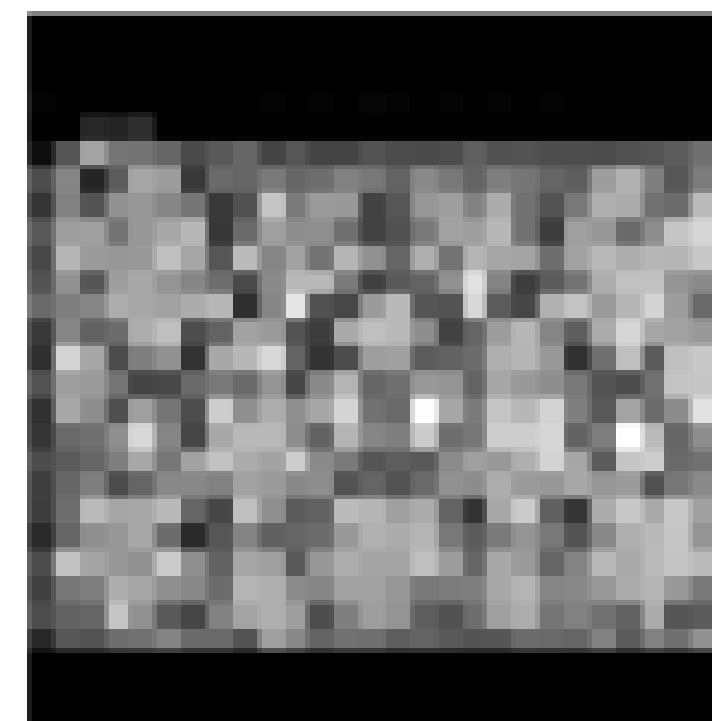
Shirt



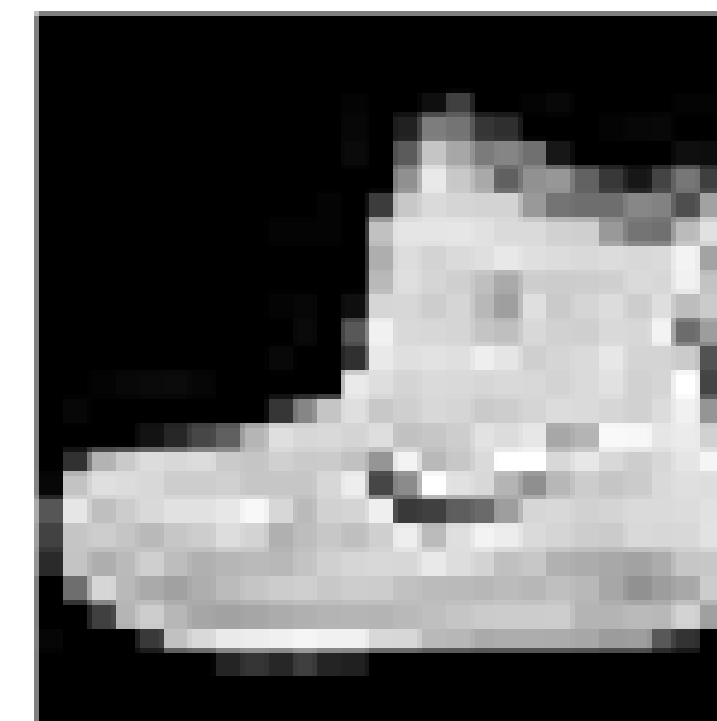
Sneaker



Bag



Ankle boot



Tensor Shape

단계	Shape	의미
입력	[64, 1, 28, 28]	이미지 64장, 채널 1개
Flatten	[64, 784]	한 장의 픽셀을 펼침
Linear + ReLU	[64, 512]	은닉 특성 512개
Linear	[64, 10]	클래스 점수 10개

배치 크기는 유지되고 각 표본의 특성 차원이 바뀜.



Model A · 기본 신경망

```
1 def make_baseline():
2     return nn.Sequential(
3         nn.Flatten(),
4         nn.Linear(784, 512),
5         nn.ReLU(),
6         nn.Linear(512, 10)
7     )
8 seed_all()
9 model_a = make_baseline().to(DEVICE)
10 h = x
11 for layer in model_a:
12     h = layer(h)
13     print(type(layer).__name__, tuple(h.shape))
14 print('Parameters:', sum(p.numel() for p in model_a.parameters()))
```

```
Flatten (64, 784)
Linear (64, 512)
ReLU (64, 512)
Linear (64, 10)
Parameters: 407050
```

Dropout 없이 기본 MLP를
먼저 만듦.

Flatten·Linear·ReLU를
순서대로 연결함.

출력은 확률 변환 전의
logits임.



예측과 손실

```

criterion = nn.CrossEntropyLoss()
def train_one_epoch(model, loader, optimizer):
    model.train()
    total_loss, correct, count = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        output = model(x)           # Forward: [B, 10]
        loss = criterion(output, y)  # scalar
        loss.backward()             # gradient
        optimizer.step()            # parameter update
        total_loss += loss.item() * len(y)
        correct += (output.argmax(1) == y).sum().item()
        count += len(y)
    return total_loss/count, correct/count

```

모델 밖에서
CrossEntropyLoss를 정의함.

예측 점수와 정답을 비교해
배치 평균 손실을 구함.

B는 현재 배치의 표본 수임.
output: [B, 10]
y: [B], loss: 스칼라

Keras에서 같은 logits를 받을 때는 from_logits=True로 설정함.
확률을 전달할 때는 from_logits=False로 설정함.

1 epoch 학습

```
criterion = nn.CrossEntropyLoss()
def train_one_epoch(model, loader, optimizer):
    model.train()
    total_loss, correct, count = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        optimizer.zero_grad()
        output = model(x)           # Forward: [B, 10]
        loss = criterion(output, y)  # scalar
        loss.backward()             # gradient
        optimizer.step()            # parameter update
        total_loss += loss.item() * len(y)
        correct += (output.argmax(1) == y).sum().item()
        count += len(y)
    return total_loss/count, correct/count
```

한 번 호출하면
학습 600장을 모두 사용함.

64장 × 9배치
+ 24장 × 1배치임.

backward(): 기울기 계산
step(): 배치마다 갱신함.

전체 10배치 완료
= 1 epoch 학습임.

C03은 가중치 갱신만 수행함. 성능 측정과 반환은 다음 C04가 담당함.

데이터 전체 평가

```
1 def evaluate(model, loader):
2     was_training = model.training
3     model.eval()
4     total_loss, correct, count = 0.0, 0, 0
5     with torch.no_grad():
6         for x, y in loader:
7             x, y = x.to(DEVICE), y.to(DEVICE)
8             output = model(x)
9             total_loss += criterion(output, y).item() * len(y)
10            correct += (output.argmax(1) == y).sum().item()
11            count += len(y)
12    model.train(was_training)
13    return total_loss/count, correct/count
14 print('Before training:', evaluate(model_a, val_loader))
```

Before training: (2.3015359020233155, 0.122)

전달받은 데이터 전체를
한 번 평가함.

학습 600장 또는
검증 1,000장을 사용함.

eval() + no_grad()
가중치를 갱신하지 않음.

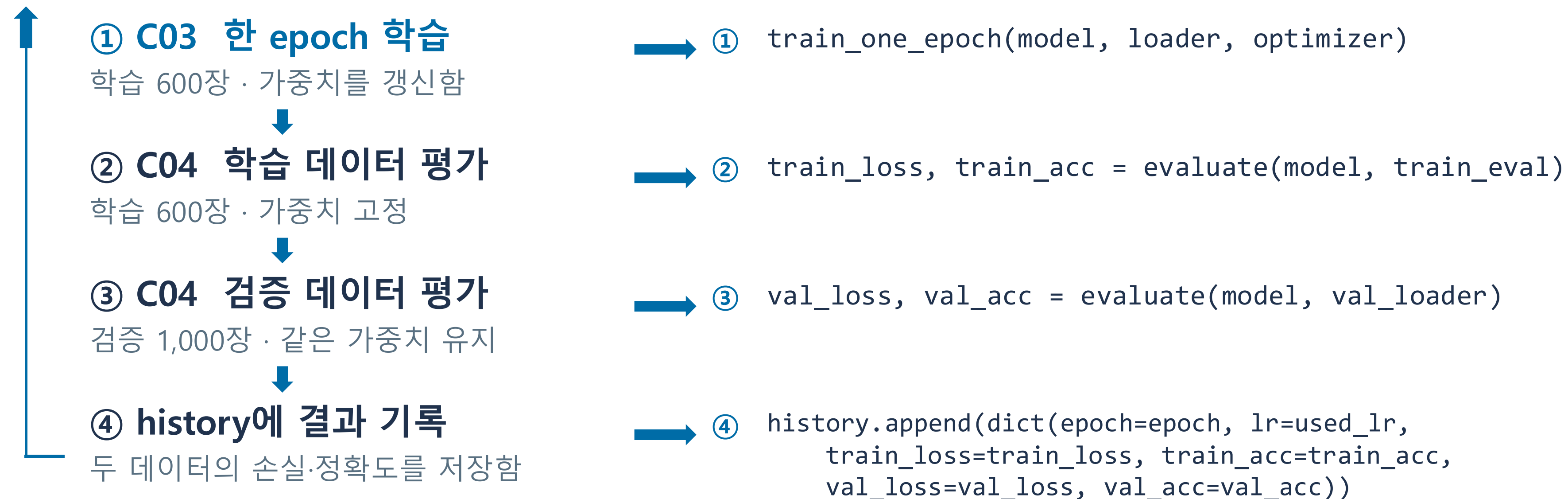
반환값: 평균 손실·정확도
평가 횟수는 학습 epoch에
더하지 않음.

배치 평균 손실 × 표본 수를 합산한 뒤 전체 표본 수로 나눔.
다음 C05에서는 C03으로 학습하고, 이 C04 함수를 두 번 호출해 학습·검증 성능을 측정함.

반복 학습과 기록

C03으로 한 번 학습하고 C04로 평가할 수 있게 됐음. C05는 이 순서를 매 epoch 반복함.

C05 fit() · for epoch in range(1, epochs + 1):



①~④ 한 묶음 = 1 epoch 학습 + 평가 2회 + 기록

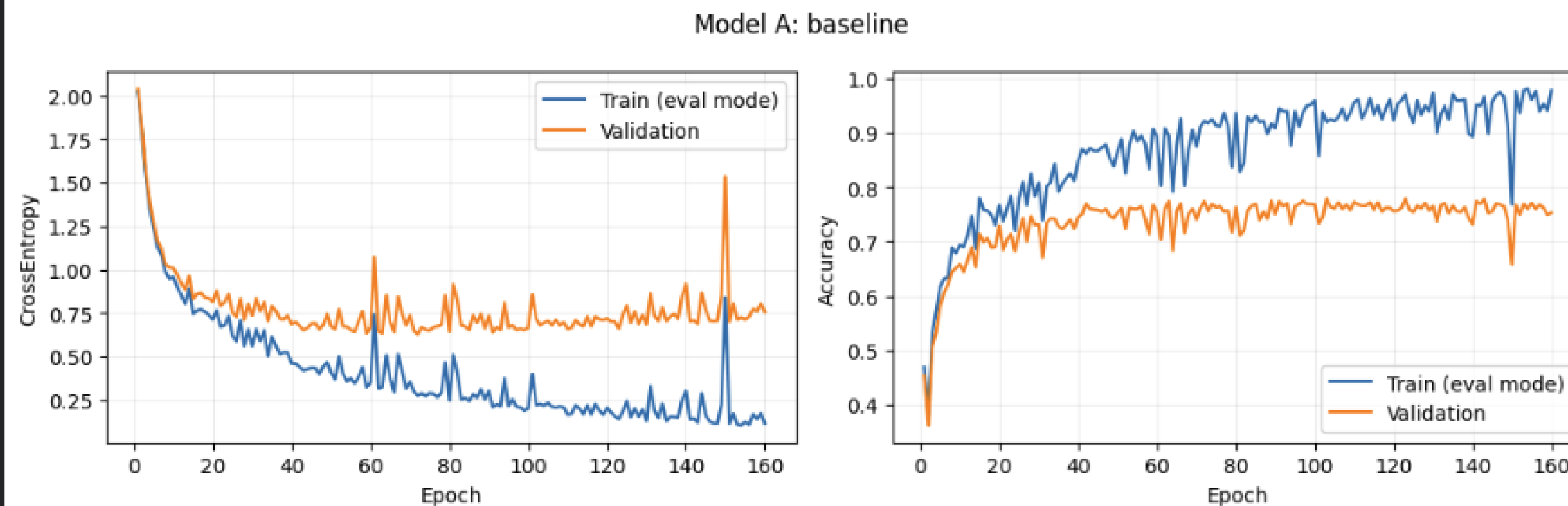
160 epoch의 기록을 모으면 다음 장의 학습·검증 곡선이 됨.
최저 검증 손실의 가중치 저장·복원은 35쪽에서 이어서 확인함.

Overfitting · 기준 모델 결과

```

1 seed_all()
2 model_a = make_baseline().to(DEVICE)
3 initial_a = copy.deepcopy(model_a.state_dict())
4 opt_a = torch.optim.SGD(model_a.parameters(), lr=LR, momentum=0)
5 results = {'baseline': fit(model_a, opt_a)}
6 report('A / no dropout', results['baseline'])
7 plot_history(results['baseline'], 'Model A: baseline')
    
```

A / no dropout: epoch 160, Train(eval) loss=0.1162, acc=97.83%
 Validation loss=0.7579, acc=75.30%
 Best validation epoch=72, loss=0.6286, time=23.02s



Dropout 없는 모델을
160 epoch 학습한 결과임.

학습 손실은 낮아지지만
검증 손실은 같은 정도로
개선되지 않음.

마지막 검증 손실: 0.7579
 최저 검증 손실: 0.6286
 최저 시점: 72 epoch임.

학습 데이터에 지나치게
맞추는 과적합을 의심함.

진단과 조절

관찰	먼저 점검할 것	확인할 부분
둘 다 잘 줄지 않음	입력·정답·갱신·학습률	Learning rate
크게 흔들림	학습률·배치·평가 조건	Scheduler
학습만 좋아짐	분할·표본 수·과적합	L2·Dropout
검증 개선이 멈춤	요동인지 지속 추세인지	Early stopping

갱신 방식은 학습률로, 과도한 맞춤은 데이터·모델·정규화로 점검함.

Learning Rate

$$w_{\text{new}} = w - \eta \times \partial L / \partial w$$

$w=2$, $\partial L / \partial w=0.5$ 일 때

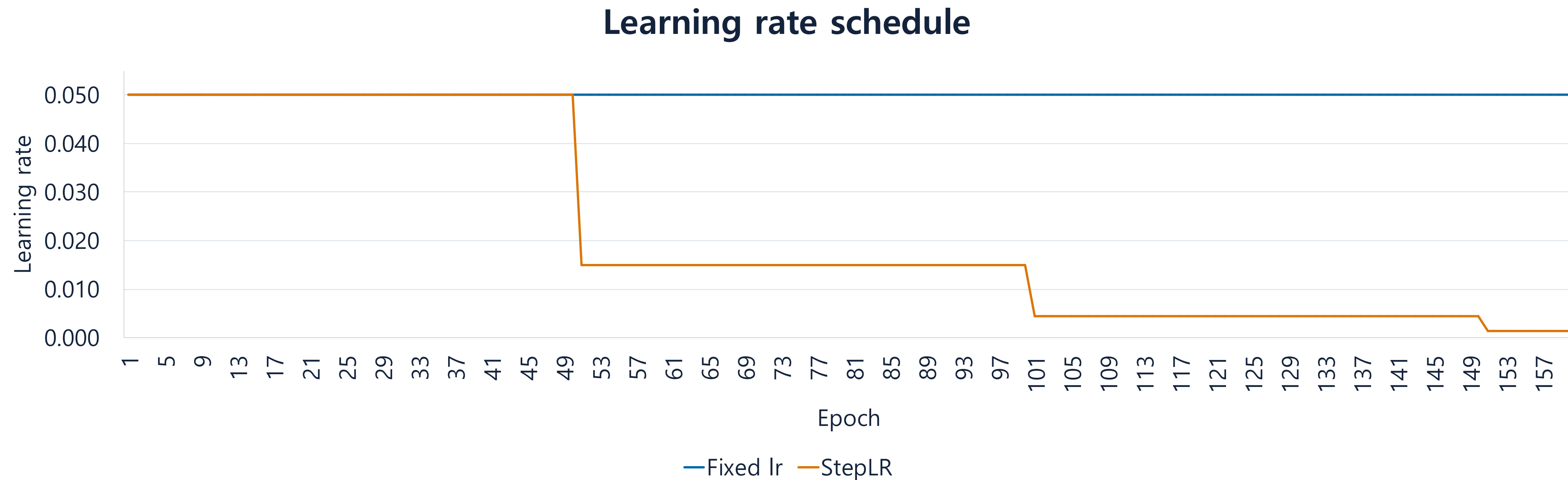
$\eta=0.1$: $w_{\text{new}}=1.95$ / $\eta=0.01$: $w_{\text{new}}=1.995$

큰 폭은 요동을, 작은 폭은 느린 학습을 유발할 수 있음.

η 는 한 번의 갱신 크기에 영향을 주는 학습률임.(lr)

Scheduler

학습률을 계속 고정할 수도 있고, 정한 시점마다 줄여 갱신 폭을 낮출 수도 있음.



예: 50 epoch마다 0.3배 함. $0.05 \rightarrow 0.015 \rightarrow 0.0045 \rightarrow 0.00135$ 임.

Scheduler 코드와 출력

```
1 parameter = nn.Parameter(torch.tensor(1.0))
2 toy_opt = torch.optim.SGD([parameter], lr=LR)
3 toy_scheduler = torch.optim.lr_scheduler.StepLR(toy_opt, step_size=50, gamma=0.3)
4 ✓ for epoch in range(1, EPOCHS+1):
5     used_lr = toy_opt.param_groups[0]['lr']
6     toy_opt.zero_grad(); (parameter*0).backward(); toy_opt.step()
7     toy_scheduler.step()
8 ✓ if epoch in [1, 50, 51, 100, 101, 150, 151]:
9     print(epoch, f'used lr={used_lr:.5f}')
```

```
1 used lr=0.05000
50 used lr=0.05000
51 used lr=0.01500
100 used lr=0.01500
101 used lr=0.00450
150 used lr=0.00450
151 used lr=0.00135
```

50 epoch까지 0.05000임.
51 epoch부터 0.01500임.

used_lr는 이번 학습에 쓴
학습률을 먼저 기록함.

optimizer.step() 뒤에
scheduler.step()을 호출함.

기울기 0으로 스케줄러만 확인함.
성능 비교 실험은 아님.

최적화와 일반화

구분	현재 질문	조절의 방향
최적화	학습 손실을 잘 줄이고 있는가?	학습률·Scheduler를 조절함
일반화	새 데이터에도 잘 적용되는가?	과도한 맞춤을 줄임

학습 손실을 낮추는 것과 새 데이터에 잘 맞는 것은 함께 확인해야 함.

과적합 완화 방법

조절 대상	방법	바뀌는 것
학습 데이터	추가 수집·데이터 증강	학습에서 접하는 표본의 다양성
모델 규모	은닉층·유닛 수 축소	학습 가능한 관계의 복잡도
가중치 크기	L1·L2 정규화	큰 가중치에 대한 벌점
학습 중 활성값	Dropout	매번 사용할 정보의 일부
학습 기간	Early stopping	검증 개선이 멈춘 뒤 학습량

정규화(Regularization): 일반화를 돕기 위해 학습에 제약을 줌.

L1 / L2

가중치 크기에 벌점을 주는 방법임.

방법	가중치 벌점	주된 성질
L1	$\lambda \sum w_i $	일부 계수가 0이 되는 해를 유도할 수 있음
L2	$(\lambda/2) \sum w_i^2$	큰 가중치에 더 큰 벌점을 줌
Elastic Net	$L1 + L2$	두 벌점의 성질을 함께 사용함

L2의 선택 기준

설명용 후보	데이터 손실	가중치 제곱합	총손실 ($\lambda=0.1$)
A: $w=(1, 1)$ $\lambda=0.1$	0.4	$1^2+1^2=2$	$0.4+0.05\times 2=0.5$
B: $w=(3, -1)$ $\lambda=0.1$	0.4	$3^2+(-1)^2=10$	$0.4+0.05\times 10=0.9$

예측 손실에 가중치 크기의 비용을 더함. λ 가 너무 크면 필요한 관계도 배우기 어려움.

L2 Penalty

$$L_{\text{total}} = L_{\text{data}} + (\lambda/2) \sum_i w_i^2$$

$$\partial L_{\text{total}} / \partial w = \partial L_{\text{data}} / \partial w + \lambda w$$

$$w_{\text{new}} = w - \eta (\partial L_{\text{data}} / \partial w + \lambda w)$$

$$w_{\text{new}} = (1 - \eta \lambda) w - \eta \partial L_{\text{data}} / \partial w$$

η : 학습률 / λ : 벌점 강도. 기본 SGD, momentum=0에서 유도한 식임.



Weight Decay 코드와 출력

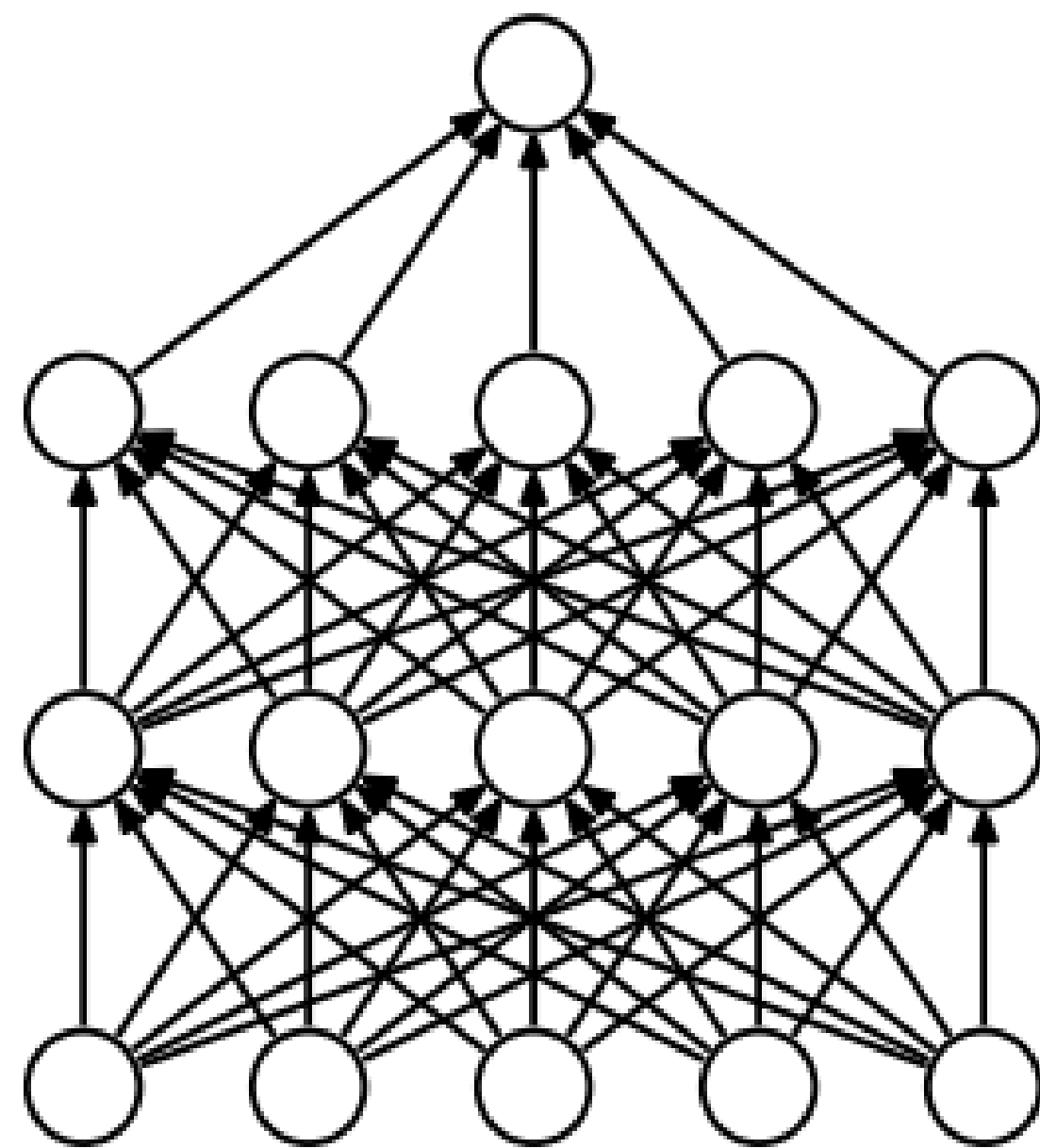
```
1 w = nn.Parameter(torch.tensor([2.0, -2.0]))
2 opt = torch.optim.SGD([w], lr=0.1, weight_decay=0.2, momentum=0)
3 w.grad = torch.zeros_like(w)
4 opt.step()
5 print('Weight decay only:', w.detach().tolist()) # 1.96, -1.96
6 print('For w=2, data gradient=0.5:', 2 - 0.1*(0.5 + 0.2*2))
```

```
Weight decay only: [1.9600000381469727, -1.9600000381469727]
For w=2, data gradient=0.5: 1.91
```

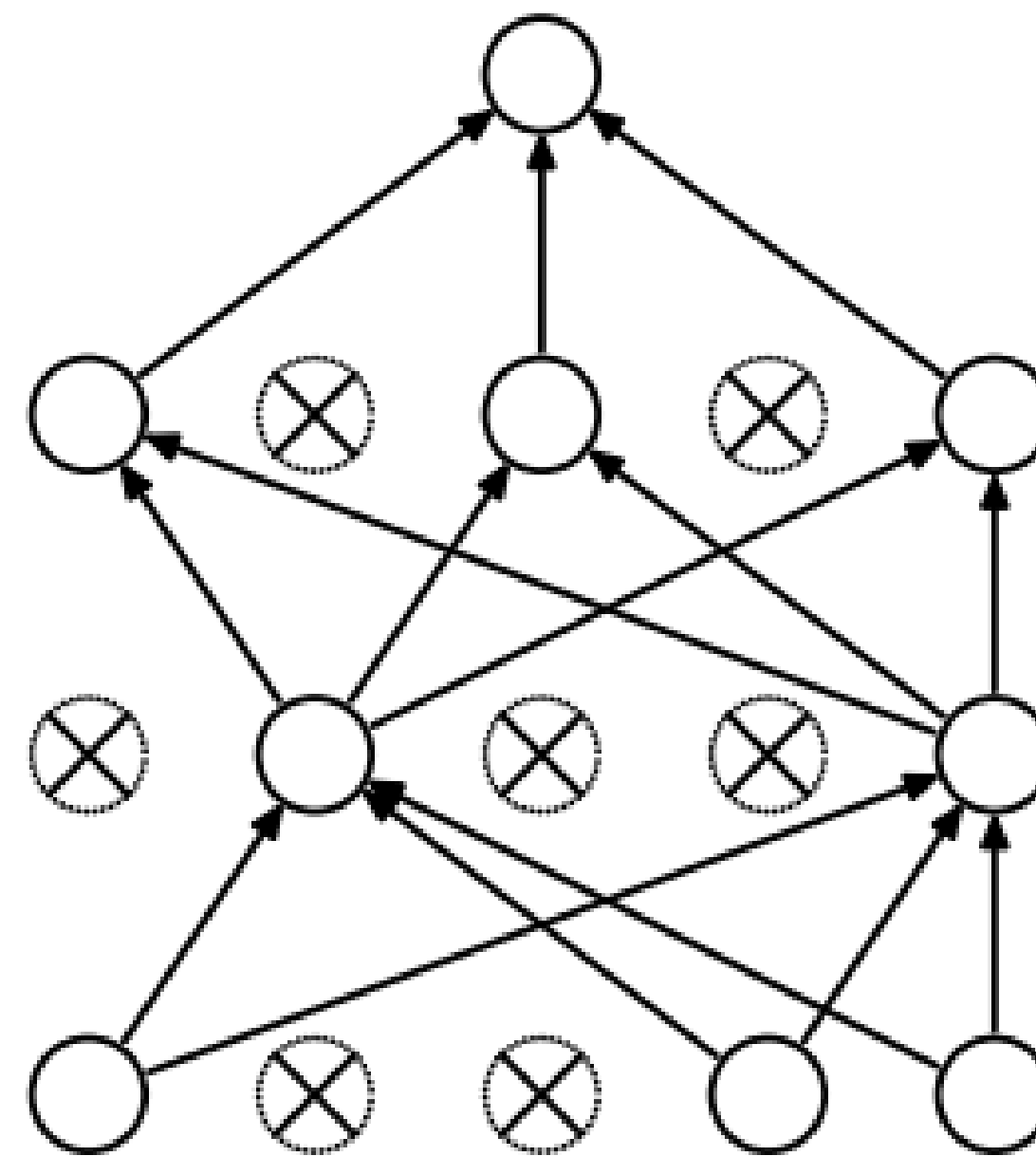
기울기 0: 2와 -2의 절댓값이 모두 1.96으로 줄어듦.
데이터 손실의 기울기 0.5까지 반영하면 w=2는 1.91이 됨.

Dropout

또 다른 방법은 학습할 때마다 사용할 활성값의 일부를 무작위로 바꾸는 것임.



(a) Standard Neural Net



(b) After applying dropout.

학습 중 일부 활성값을 확률 p 로 0으로 만듦.

매번 다른 일부 정보가 없어도 예측하도록 학습함.

연결과 가중치를 영구 삭제하는 작업이 아님.

p 는 제거 확률임. 우리 모델은 ReLU 뒤 한 층에 적용함.



Dropout 보정

$h=2, p=0.5$: 보정 없이 제거하면 0 또는 2임.

$$E[mh] = 0 \times p + h \times (1-p) = (1-p)h$$

학습 출력 $\tilde{h} = mh/(1-p)$, m 은 0 또는 1임.

생존 값은 $2/0.5=4$ 임. 기대값은 $0 \times 0.5 + 4 \times 0.5 = 2$ 임.

PyTorch와 Keras는 학습 시 생존값을 보정하고 평가 시 그대로 통과시킴.

Dropout 보정 방식

방식	학습 시	평가 시
PyTorch / Keras	$mh/(1-q)$	h 를 그대로 통과시킴
Dropout 제안 논문	mh	다음 층 가중치에 $(1-q)$ 를 곱함

q 는 앞 장의 제거 확률 p 와 같음. 논문에서 사용하는 p (유지 확률)와 구분하기 위한 표기임.

Train vs Evaluation

```
1  seed_all()
2  x_small = torch.ones(12)
3  drop = nn.Dropout(p=0.5)
4  drop.train()
5  print('train 1:', drop(x_small).tolist())
6  print('train 2:', drop(x_small).tolist())
7  drop.eval()
8  print('eval 1:', drop(x_small).tolist())
9  print('eval 2:', drop(x_small).tolist())
10 print('shape:', tuple(x_small.shape), '->', tuple(drop(x_small).shape))
```

```
train 1: [2.0, 2.0, 2.0, 2.0, 0.0, 2.0, 0.0, 0.0, 2.0, 2.0, 2.0, 2.0]
train 2: [0.0, 0.0, 2.0, 0.0, 2.0, 0.0, 0.0, 2.0, 2.0, 0.0, 2.0, 2.0]
eval 1: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
eval 2: [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
shape: (12,) -> (12,)
```

입력은 모두 1인 값 12개임.
p=0.5로 설정함.

train 출력은 0 또는 2임.
무작위 제거 위치가 바뀜.

eval 출력은 모두 1임.
입력을 그대로 통과시킴.

shape는 두 모드에서
모두 (12,)로 유지됨.



PyTorch / Keras

PyTorch

```
import torch
from torch import nn
x = torch.ones(12)
drop = nn.Dropout(p=0.5)
drop.train()
print(drop(x))
drop.eval()
print(drop(x))
```

TensorFlow / Keras · 비교 코드

```
import tensorflow as tf
x = tf.ones((12,))
drop = tf.keras.layers.Dropout(0.5)

print(drop(x, training=True))

print(drop(x, training=False))
```

eval() / no_grad()

설정	바꾸는 것	Dropout
model.train()	계층의 학습 동작	무작위 제거를 적용함
model.eval()	계층의 평가 동작	제거를 끄
torch.no_grad()	기울기 추적	모드를 바꾸지 않음

평가에서는 eval()과 no_grad()를 각각 설정함.

학습·평가 제어

개념	PyTorch	TensorFlow / Keras
학습 동작	model.train() 후 호출	drop(x, training=True)
평가 동작	model.eval() 후 호출	drop(x, training=False)
확률의 의미	p = 제거 확률	rate = 제거 확률

두 프레임워크에서 무작위 마스크가 같을 필요는 없음.

기울기 기록 방식

구분	PyTorch	TensorFlow 직접 작성 루프
학습 기록	기본 grad 모드에서 requires_grad에 따라 기록함	tf.GradientTape() 범위에서 필요한 연산 기록함
평가 호출	eval() + no_grad()	활성 GradientTape 밖에서 model(x, training=False)
계층 동작	train() / eval()	training=True / False

계층의 학습·평가 동작과 미분용 기록은 별개의 설정임.

Model B - Dropout 추가

```
1  def make_dropout(p):
2      return nn.Sequential(
3          nn.Flatten(),
4          nn.Linear(784, 512),
5          nn.ReLU(),
6          nn.Dropout(p=p),
7          nn.Linear(512, 10)
8      )
9  seed_all()
10 model_b = make_dropout(p=0.5).to(DEVICE)
11 for a, b in zip(initial_a.values(), model_b.state_dict().values()):
12     assert torch.equal(a, b)
13 h = x
14 for layer in model_b:
15     h = layer(h); print(type(layer).__name__, tuple(h.shape))
16 print('Parameters:', sum(p.numel() for p in model_b.parameters()))
```

```
Flatten (64, 784)
Linear (64, 512)
ReLU (64, 512)
Dropout (64, 512)
Linear (64, 10)
Parameters: 407050
```

기존 ReLU 뒤에
Dropout(p=0.5)을 추가함.

Linear의 초기 가중치를
기존 모델과 같게 맞춤.

Dropout 뒤의 shape는
[64, 512]로 유지됨.

파라미터 수는
407,050개로 동일함.

비교 조건

고정 조건	설정
데이터	학습 600 / 검증 1,000, 같은 표본
모델	784-512-10, 같은 초기 가중치
학습	SGD, momentum=0, batch 64, 160 epoch
초기 학습률	0.05
변경 조건	A: Dropout 없음 / B: p=0.5

공통 평가: epoch 종료 후 eval 모드의 손실·정확도를 기록함.



최적 가중치의 저장과 복원

```
if val_loss < best_loss:
    best_loss, best_epoch, bad = val_loss, epoch, 0
    best_state = copy.deepcopy(model.state_dict())
else:
    bad += 1
    if scheduler is not None:
        scheduler.step() # next epoch uses the updated lr
    if patience is not None and bad >= patience:
        break
final_state = copy.deepcopy(model.state_dict())
model.load_state_dict(best_state)
return dict(history=history, final=history[-1], best=history[best_epoch-1],
            best_state=best_state, final_state=final_state,
            stop_epoch=history[-1]['epoch'], seconds=time.perf_counter()
            -started)
```

검증 손실이 낮아지면
best_loss와 epoch를 갱신함.

deepcopy로 해당 시점의
가중치를 따로 보관함.

학습 종료 후 best_state를
복원하고 기록을 반환함.

Scheduler·조기 종료는
설정된 경우에만 적용함.

Dropout 비교 실행

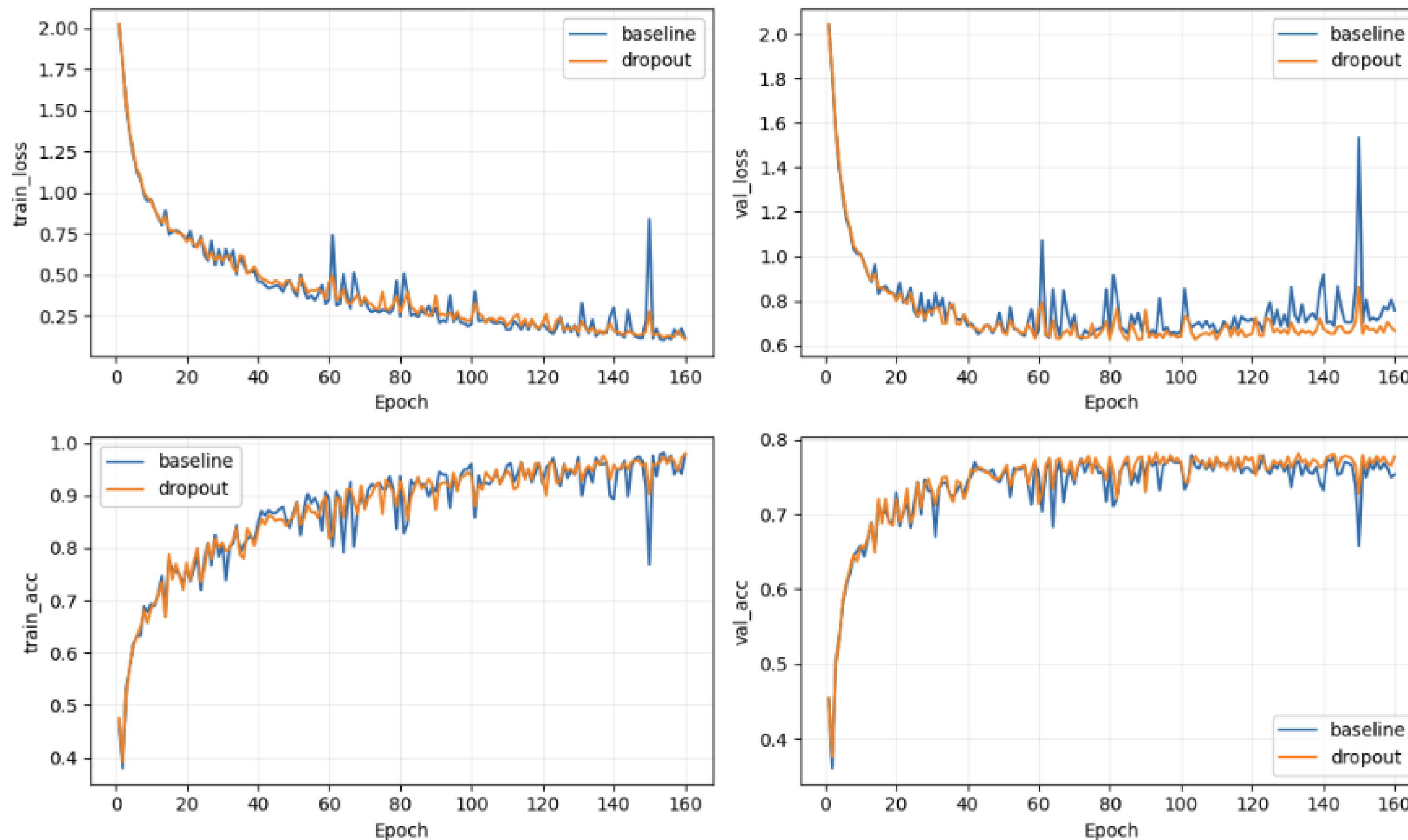
```
1 seed_all()
2 model_b = make_dropout(p=0.5).to(DEVICE)
3 opt_b = torch.optim.SGD(model_b.parameters(), lr=LR, momentum=0)
4 results['dropout'] = fit(model_b, opt_b)
5 report('B / dropout p=0.5', results['dropout'])
6 fig, axes = plt.subplots(2, 2, figsize=(11, 7))
7 for ax, key in zip(axes.flat, ['train_loss', 'val_loss', 'train_acc',
    'val_acc']):
8     for name in ['baseline', 'dropout']:
9         h = results[name]['history']
10        ax.plot([r['epoch'] for r in h], [r[key] for r in h], label=name)
11        ax.set(xlabel='Epoch', ylabel=key); ax.legend(); ax.grid(alpha=.2)
12 fig.suptitle('A/B comparison: train and validation evaluated in eval mode')
13 plt.tight_layout(); plt.show()
```

```
B / dropout p=0.5: epoch 160, Train(eval) loss=0.1104, acc=98.00%
Validation loss=0.6660, acc=77.70%
Best validation epoch=85, loss=0.6237, time=17.36s
```

모델을 새로 만든 뒤 같은 SGD 설정으로 학습함. 비교에서는 Dropout p만 바꿈.
출력의 epoch 160은 마지막 시점이며, Best validation epoch=85는 최저 손실 시점임.

손실과 정확도 비교

A/B comparison: train and validation evaluated in eval mode



위: 학습·검증 손실임.
아래: 학습·검증 정확도임.

파란색: baseline임.
주황색: dropout임.

모든 Train 지표도
eval 모드에서 다시 측정함.

학습 정확도가 반드시
낮아지는 것은 아님.
검증 추세와 함께 해석함.



A/B 실행 결과 해석

Model A · Dropout 없음

```
A / no dropout: epoch 160, Train(eval) loss=0.1162, acc=97.83%  
Validation loss=0.7579, acc=75.30%  
Best validation epoch=72, loss=0.6286, time=23.02s
```

Model B · Dropout $p=0.5$

```
B / dropout  $p=0.5$ : epoch 160, Train(eval) loss=0.1104, acc=98.00%  
Validation loss=0.6660, acc=77.70%  
Best validation epoch=85, loss=0.6237, time=17.36s
```

마지막 검증 정확도는 75.30%에서 77.70%로 2.40%p 높아짐.
단일 seed의 관찰이며, 다른 데이터·초기값에서도 같은 결과를 보장하지 않음.

Scheduler / Weight Decay

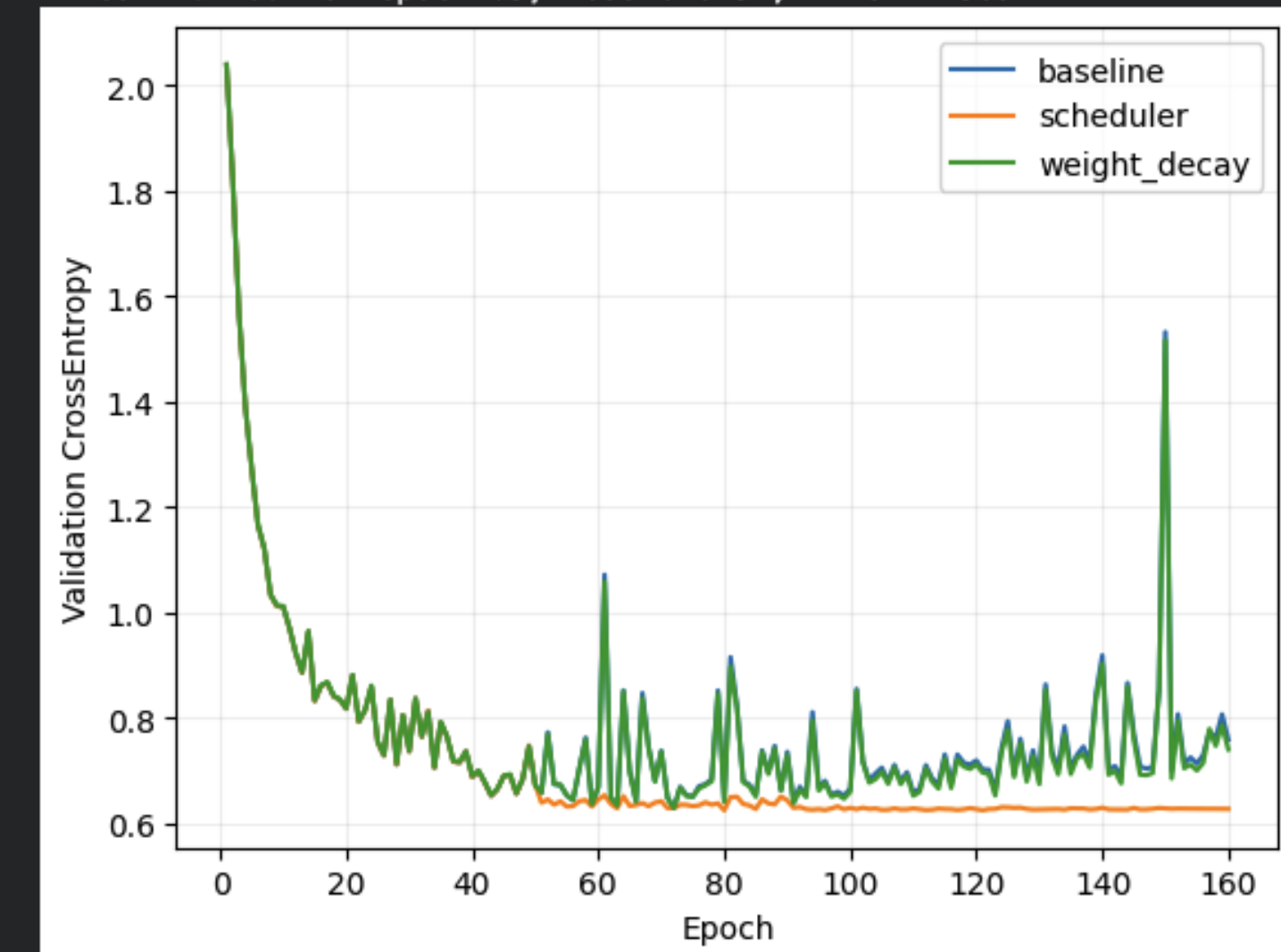
```

1  seed_all()
2  model_s = make_baseline().to(DEVICE)
3  opt_s = torch.optim.SGD(model_s.parameters(), lr=LR, momentum=0)
4  scheduler = torch.optim.lr_scheduler.StepLR(opt_s, step_size=50, gamma=0.3)
5  results['scheduler'] = fit(model_s, opt_s, scheduler=scheduler)
6  seed_all()
7  model_w = make_baseline().to(DEVICE)
8  weights = [v for name, v in model_w.named_parameters() if name.endswith
              ('weight')]
9  biases = [v for name, v in model_w.named_parameters() if name.endswith('bias')]
10 opt_w = torch.optim.SGD([{'params': weights, 'weight_decay': 0.001},
11                           {'params': biases, 'weight_decay': 0.0}], lr=LR,
                           momentum=0)
12 results['weight_decay'] = fit(model_w, opt_w)
13 for name in ['baseline', 'scheduler', 'weight_decay', 'dropout']:
14     report(name, results[name])
15 for name in ['baseline', 'scheduler', 'weight_decay']:
16     h = results[name]['history']
17     plt.plot([r['epoch'] for r in h], [r['val_loss'] for r in h], label=name)
18 plt.xlabel('Epoch'); plt.ylabel('Validation CrossEntropy'); plt.legend(); plt.
    grid(alpha=.2); plt.show()
  
```

각 조건은 새 기준 모델에서 시작함.
 Scheduler: StepLR(50, 0.3)임.
 Weight decay: 0.001, bias 제외임.

```

baseline: epoch 160, Train(eval) loss=0.1162, acc=97.83%
Validation loss=0.7579, acc=75.30%
Best validation epoch=72, loss=0.6286, time=23.02s
scheduler: epoch 160, Train(eval) loss=0.2649, acc=93.00%
Validation loss=0.6267, acc=77.50%
Best validation epoch=80, loss=0.6240, time=14.50s
weight_decay: epoch 160, Train(eval) loss=0.1276, acc=97.33%
Validation loss=0.7398, acc=75.50%
Best validation epoch=72, loss=0.6280, time=14.94s
dropout: epoch 160, Train(eval) loss=0.1104, acc=98.00%
Validation loss=0.6660, acc=77.70%
Best validation epoch=85, loss=0.6237, time=17.36s
  
```



Early Stopping

patience=2: 최저 검증 손실을 2 epoch 연속 갱신하지 못하면 중단함.

설명용 epoch	검증 손실	판단
1	0.60	현재 최저 상태를 저장함
2	0.50	개선됨. 대기 횟수를 0으로 함
3	0.53	최저보다 나쁨. 1회 대기함
4	0.52	최저보다 나쁨. 2회 대기 후 중단함

설명용 표는 patience=2임. 다음 장 조기 종료 실험 실제 실행은 patience=8을 사용함.

조기 종료와 가중치 복원

```
1 seed_all()
2 model_e = make_baseline().to(DEVICE)
3 opt_e = torch.optim.SGD(model_e.parameters(), lr=LR, momentum=0)
4 results['early_stop'] = fit(model_e, opt_e, patience=8)
5 r = results['early_stop']
6 print('Stop epoch:', r['stop_epoch'], 'Restore epoch:', r['best']['epoch'])
7 print('Restored validation:', evaluate(model_e, val_loader))
8 print('Continued baseline best:', results['baseline']['best']['epoch'],
9       results['baseline']['best']['val_loss'])
10 assert abs(evaluate(model_e, val_loader)[0] - r['best']['val_loss']) < 1e-6
```

```
Stop epoch: 51 Restore epoch: 43
Restored validation: (0.6515850868225098, 0.76)
Continued baseline best: 72 0.6285773968696594
```

patience=8로 학습함.

51 epoch에서 중단함.
43 epoch의 가중치를 복원함.

복원 후 검증 정확도: 76%임.
검증 손실: 약 0.6516임.

끝까지 학습한 기준 모델은
72 epoch에 더 낮은 손실을
기록함.



모델 선택과 저장

```
1 candidates = ['baseline', 'scheduler', 'weight_decay', 'dropout']
2 selected = min(candidates, key=lambda name: results[name]['best']['val_loss'])
3 print('Selected:', selected, 'epoch:', results[selected]['best']['epoch'])
4 torch.save(results[selected]['best_state'], OUT/'selected_weights.pt')
5 config = dict(seed=SEED, train_size=600, val_size=1000, epochs=EPOCHS,
6               batch_size=BATCH, lr=LR, optimizer='SGD momentum=0',
7               model='784-512-10', dropout_p=0.5, weight_decay=0.001,
8               scheduler_step_size=50, scheduler_gamma=0.3, patience=8,
9               device=str(DEVICE), torch_version=str(torch.__version__),
10              selection='minimum validation CrossEntropy',
11              test_used=False, selected=selected)
12 def save_results():
13     clean = {k:{a:b for a,b in v.items() if a not in ['best_state',
14              'final_state']}}
15     for k,v in results.items():
16         (OUT/'results.json').write_text(json.dumps(dict(config=config,
17              results=clean), indent=2))
18 save_results()
19 print('Saved:', OUT/'results.json')
```

Selected: dropout epoch: 85
Saved: results/results.json

네 후보의 최저 검증 손실을
같은 기준으로 비교함.

이번 후보 집합에서는
Dropout p=0.5의
85 epoch를 선택함.

선택한 가중치와
실험 기록을 저장함.

뒤의 p=0.2·0.7 실험은
이 후보 목록에 포함되지 않음.

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Nitish Srivastava
Geoffrey Hinton
Alex Krizhevsky
Ilya Sutskever
Ruslan Salakhutdinov
*Department of Computer Science
University of Toronto
10 Kings College Road, Rm 3302
Toronto, Ontario, M5S 3G4, Canada.*

NITISH@CS.TORONTO.EDU
HINTON@CS.TORONTO.EDU
KRIZ@CS.TORONTO.EDU
ILYA@CS.TORONTO.EDU
RSALAKHU@CS.TORONTO.EDU

Abstract

Deep neural nets with a large number of parameters are very powerful machine learning systems. However, overfitting is a serious problem in such networks. Large networks are also slow to use, making it difficult to deal with overfitting by combining the predictions of many different large neural nets at test time. Dropout is a technique for addressing this problem. The key idea is to randomly drop units (along with their connections) from the neural network during training. This prevents units from co-adapting too much. During training, dropout samples from an exponential number of different “thinned” networks. At test time,

큰 신경망의 과적합을 줄이려는
연구임.

여러 큰 모델의 예측 결합은 계산
비용이 큼.

무작위 유닛 제거와 한 모델의 평
가로 접근함.

우리 실습에서 관찰한 차이가 어떤 근거를 갖는지, 원논문의 방법과 실험 조건을 확인함.

확률 표기

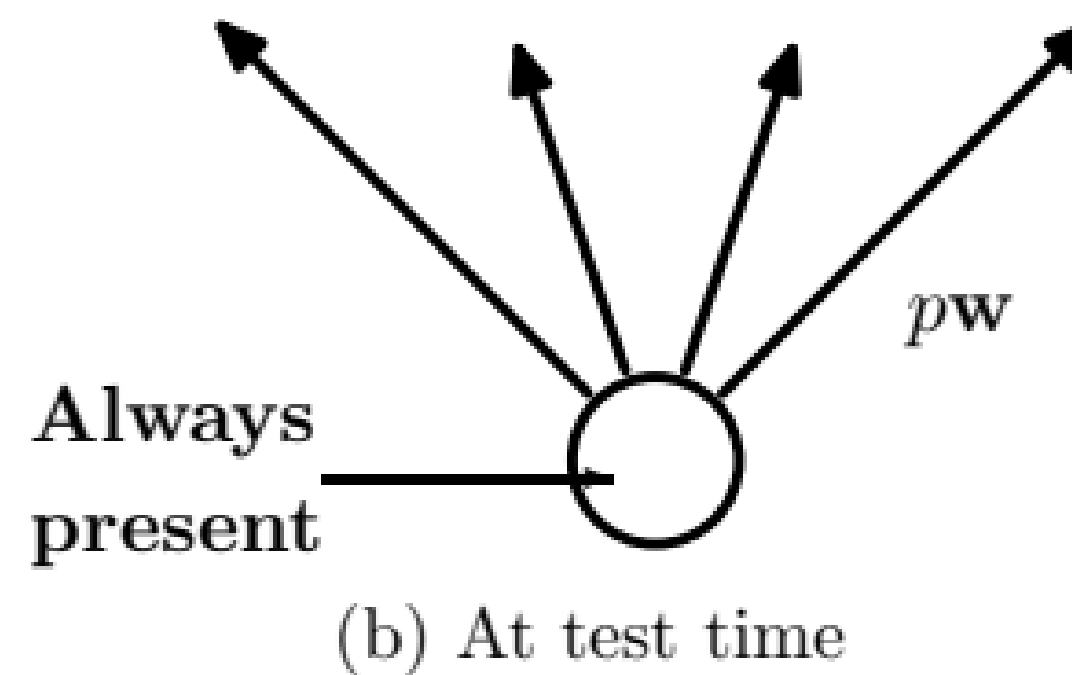
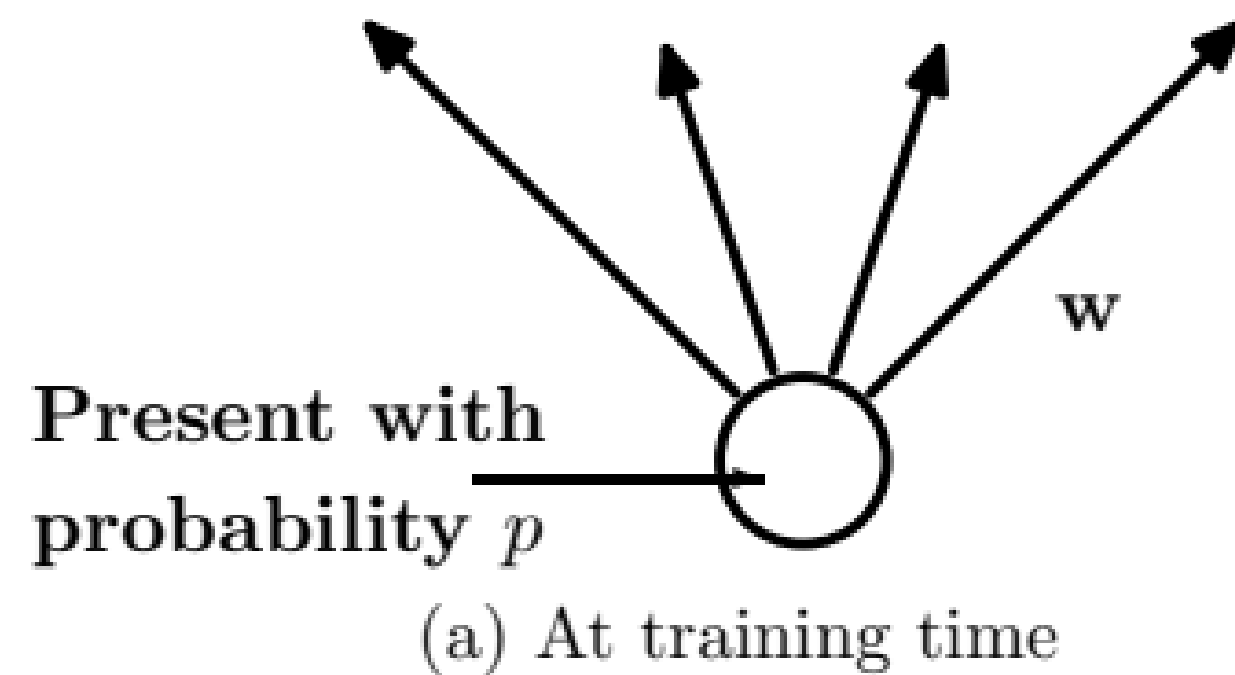
표기 연결: 제거 확률 $q = 1 -$ 원논문의 유지 확률 p 임.

원문의 p 는 유지 확률임.

PyTorch와 Keras의 p -rate는 제거 확률임.

원문은 평가 때 가중치를 보정함.

우리 코드는 학습 때 생존 활성값을 보정함.



논문 수식과 코드

$$r_j^{(l)} \sim \text{Bernoulli}(p),$$

$$\tilde{\mathbf{y}}^{(l)} = \mathbf{r}^{(l)} * \mathbf{y}^{(l)},$$

$$z_i^{(l+1)} = \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^l + b_i^{(l+1)},$$

$$y_i^{(l+1)} = f(z_i^{(l+1)}).$$

r 은 유지 여부를 뽑는 0/1 마스크임.

$\tilde{y} = r \odot y$ 로 활성값을 가림.

가려진 $\tilde{y}$ 가 다음 Linear의 입력임.

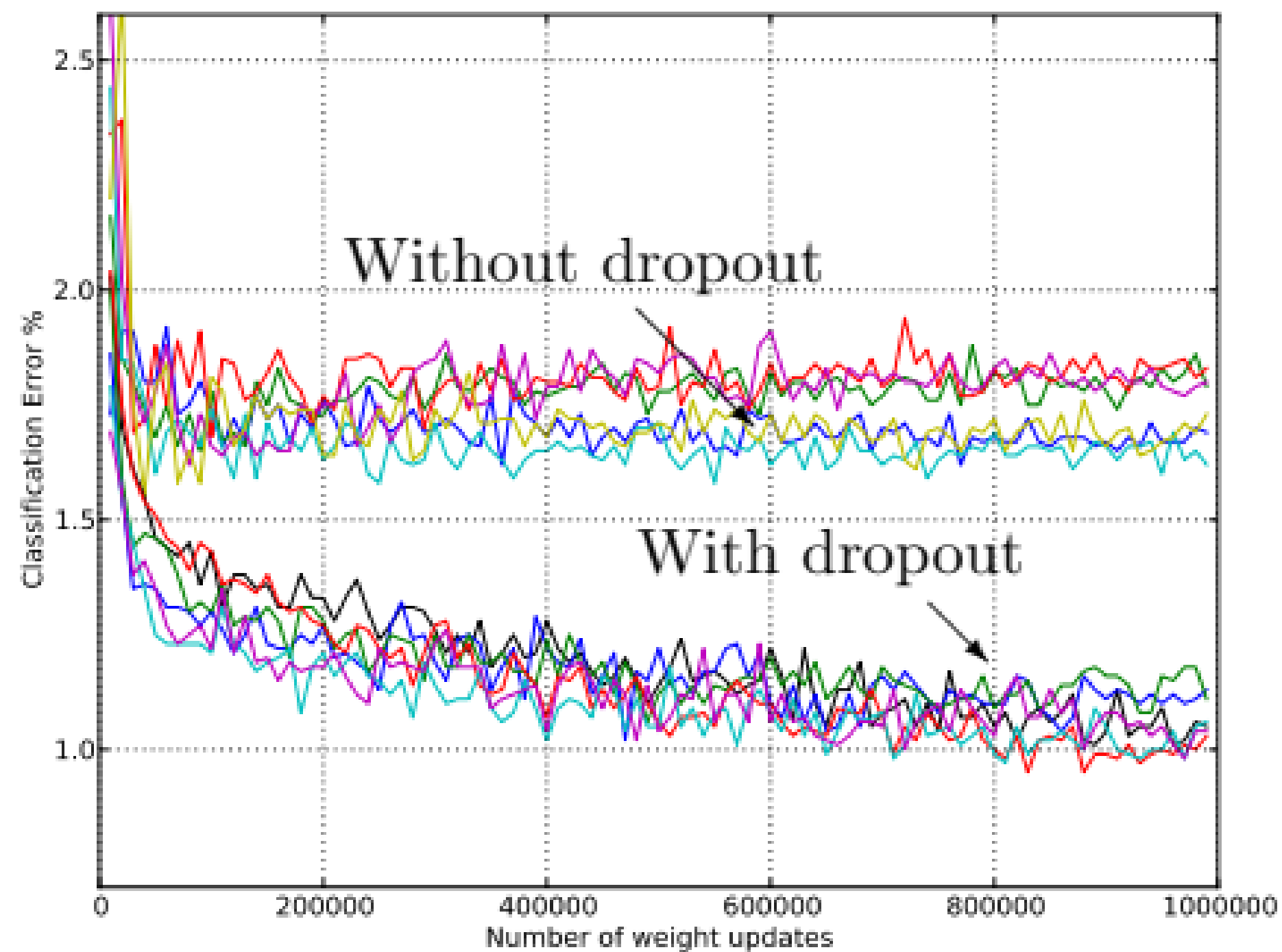
실습은 생존값에 $1/(1-q)$ 를 곱함.

논문 실험 조건

항목	Figure 4 및 Appendix B.1
데이터	MNIST 손글씨 숫자 이미지임
구조	은닉층 2·3·4개 × 층당 1,024·2,048개: 총 6구조임
유지 확률	은닉층 0.5, 입력층 0.8임
학습 조건	100만 weight update, 최종 momentum 0.95임
가중치 제약	max-norm c=2를 함께 사용함



논문 실험



데이터는 MNIST임. Fashion-MNIST가 아님.

가로축은 가중치 갱신 횟수임.

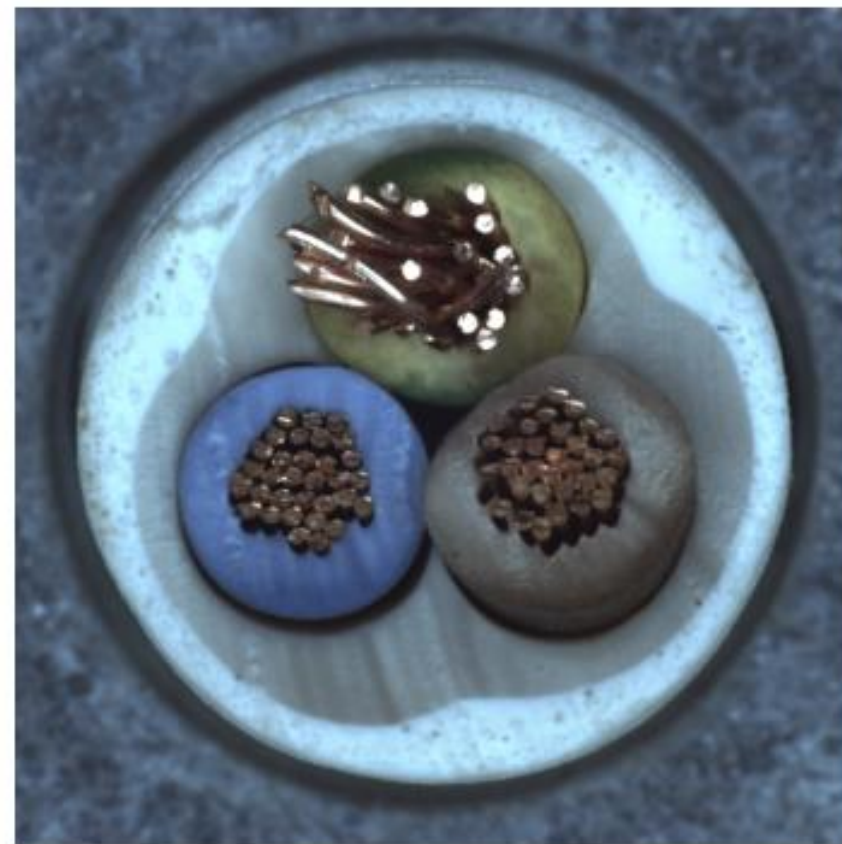
세로축은 테스트 오류율이며 낮을수록 좋음.

With / Without dropout 곡선군을 비교함.

해석의 범위

비교 항목	원문 Figure 4	이번 실습
데이터	MNIST	Fashion-MNIST
모델	깊고 넓은 여러 구조	784-512-10 MLP
조건	max-norm 등 추가 조건	고정 SGD와 한 조건씩 변경
결론 범위	논문 조건에서의 결과	작은 데이터·단일 seed의 관찰

실무의 판단 기준



결함 제품 / 정상 기준 / 예측 영역 / 정답 영역

Defective Image

Golden Image

Predicted Defect Mask

True Defect Mask



불량 누락과 정상 오탐의 비용을 구분함.

고객 대응에서는 연락 가능한 인원도 제약임.

온라인 서비스는 예측 품질과 응답 시간을 함께 봄.

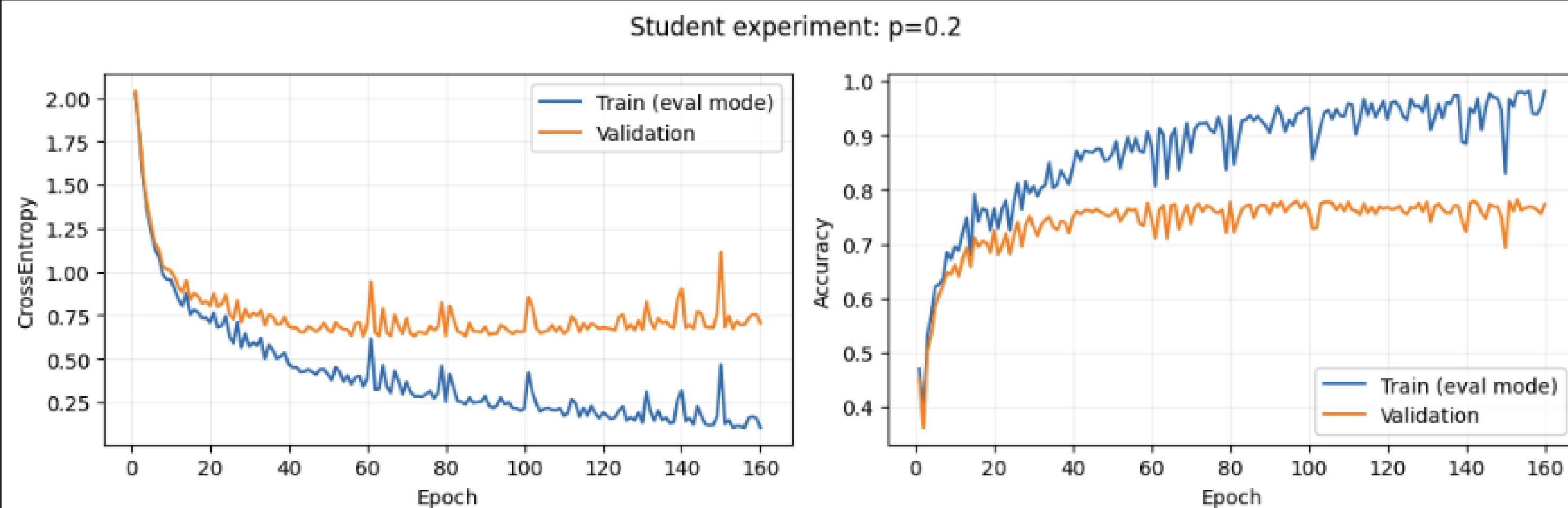
실습의 선택 기준과 실제 업무의 판단 기준을 구분함.

Dropout 비율 변경 실험

```

1 P_TRY = 0.2
2 seed_all()
3 model_try = make_dropout(p=P_TRY).to(DEVICE)
4 opt_try = torch.optim.SGD(model_try.parameters(), lr=LR, momentum=0)
5 trial_key = f'p={round(P_TRY*10):02d}'
6 results[trial_key] = fit(model_try, opt_try)
7 report(f'p={P_TRY}', results[trial_key])
8 plot_history(results[trial_key], f'Student experiment: p={P_TRY}')
9 save_results()
  
```

p=0.2: epoch 160, Train(eval) loss=0.1060, acc=98.17%
 Validation loss=0.7073, acc=77.30%
 Best validation epoch=59, loss=0.6275, time=17.66s



P_TRY만 0.2로 바꿈.
 모델은 처음부터 새로 학습함.

마지막 검증 손실: 0.7073임.
 검증 정확도: 77.30%임.

최저 검증 손실: 0.6275임.
 그 시점은 59 epoch임.

마지막 값과 최저값을
 따로 기록해 비교함.

Dropout 비율별 결과

```
1 seed_all()
2 model_07 = make_dropout(p=0.7).to(DEVICE)
3 opt_07 = torch.optim.SGD(model_07.parameters(), lr=LR, momentum=0)
4 results['p07'] = fit(model_07, opt_07)
5 for p, name in [(0.0, 'baseline'), (0.2, 'p02'), (0.5, 'dropout'), (0.7, 'p07')]:
6     report(f'p={p}', results[name])
7 save_results()
```

```
p=0.0: epoch 160, Train(eval) loss=0.1162, acc=97.83%
      Validation loss=0.7579, acc=75.30%
      Best validation epoch=72, loss=0.6286, time=23.02s
p=0.2: epoch 160, Train(eval) loss=0.1060, acc=98.17%
      Validation loss=0.7073, acc=77.30%
      Best validation epoch=59, loss=0.6275, time=17.66s
p=0.5: epoch 160, Train(eval) loss=0.1104, acc=98.00%
      Validation loss=0.6660, acc=77.70%
      Best validation epoch=85, loss=0.6237, time=17.36s
p=0.7: epoch 160, Train(eval) loss=0.1367, acc=97.33%
      Validation loss=0.6387, acc=77.70%
      Best validation epoch=85, loss=0.6110, time=15.70s
```

p=0.0, 0.2, 0.5, 0.7을
같은 조건으로 비교함.

각 조건의 epoch 160 값과
최저 검증 손실을 구분함.

이번 네 비율 중 최저 손실은
p=0.7의 85 epoch에서
0.6110임.

단일 seed의 결과이므로
항상 p=0.7이 좋다고
일반화하지 않음.

다른 문제에 적용

문제	바꿀 항목	유지할 원리
숫자 이미지	클래스 의미	검증과 고정 조건
고객 이탈	입력 특성·분할·표준화	한 조건씩 비교
수요량 예측	출력 1개·회귀 손실·시간 분할	학습·검증 분리

과제

학생용 코드 빈칸 17개를 완성함. PPT를 보고 따라 입력해도 됨.

각 셀의 참고 쪽수를 확인하고 완성 후 처음부터 실행함.

$p=0.0, 0.2, 0.5, 0.7$ 의 네 지표·곡선·최저 손실을 비교함.

실행한 ipynb와 비교 표, 선택 이유 5~7문장을 제출함.

AI를 사용했다면 관련 프롬프트도 함께 제출함.